# Qiskit Certification — Visualization Module

## Answer Key with Full Reasoning (30 Questions)

Study Group Prep Deck

August 29, 2026

# Answer Key — Overview

Full solutions with reasoning follow, one question per slide.

Companion to the 30-question “Visualization Module” practice PDF.

# Answer Key — Question 1

## Correct Answer: B

The $i$-th color in `color` is mapped to the $i$-th dataset, matching the order of `data` and `legend`

## Reasoning

`plot_histogram` pairs list-valued kwargs (`legend`, `color`) positionally with the list of counts dictionaries passed as `data`. Index 0 of `color` styles `counts1`, index 1 styles `counts2`, exactly matching the notebook's `legend` ordering.

# Answer Key — Question 2

## Correct Answer: C

Both: pass `filename` as a kwarg to `plot_histogram`, or capture its return value (a Matplotlib `Figure`) and call `.savefig()` on it

## Reasoning

The notebook explicitly demonstrates two equivalent techniques: passing `filename='new_hist.png'` as a kwarg, and separately storing `hist = plot_histogram(data)` then calling `hist.savefig('new_hist.png')`, since the function returns a standard Matplotlib `Figure`.

# Answer Key — Question 3

## Correct Answer: A

`plot_distribution` normalizes raw counts into a probability distribution before plotting; `plot_histogram` plots the raw counts as-is

## Reasoning

`plot_histogram` plots the input counts dictionary directly (bar heights = raw shot counts), while `plot_distribution` is designed to visualize a *sampled probability distribution*, so it renders normalized frequencies rather than raw integer counts.

# Answer Key — Question 4

## Correct Answer: B

$[r, \theta, \phi]$ where $\theta$ is the polar (inclination) angle from $+z$ and $\phi$ is the azimuthal angle from $+x$, both in radians

## Reasoning

With `coord_type='spherical'` the notebook specifies the ordering explicitly: $[\langle r \rangle, \theta, \phi]$, with $\theta$ the inclination from the $+z$ axis and $\phi$ the azimuth from the $+x$ axis, both given in radians (not degrees).

# Answer Key — Question 5

## Correct Answer: B

`cartesian`

## Reasoning

The notebook states the default `coord_type` is Cartesian $[\langle x \rangle, \langle y \rangle, \langle z \rangle]$; spherical input must be requested explicitly.

# Answer Key — Question 6

## Correct Answer: B

Two separate points, each on the *surface* of its own sphere, because the qubits are unentangled (the state is a simple product state)

## Reasoning

$H \otimes X$ applied to $|00\rangle$ produces the product state $|+\rangle \otimes |1\rangle$ — no entanglement. Each qubit therefore has a pure reduced state, plotted as a point exactly on its own sphere's surface (comment in the notebook: "the two qubits are unentangled hence can be represented as separate points on the surface").

# Answer Key — Question 7

## Correct Answer: B

Each qubit's reduced density matrix is $\rho_i = I/2$, giving $\langle X \rangle = \langle Y \rangle = \langle Z \rangle = 0$, i.e. the Bloch vector $(0, 0, 0)$

## Reasoning

Tracing out qubit 2 from the maximally entangled Bell state leaves the maximally mixed reduced state $\rho_i = I/2$ for qubit 1 (and vice versa). All three Pauli expectation values vanish, so the Bloch vector has zero length and plots exactly at the sphere's center — this is the textbook signature of maximal entanglement, not a plotting limitation.

# Answer Key — Question 8

## Correct Answer: B

Reverses the qubit display order (e.g. draws $q_1$ before $q_0$), inverting Qiskit's default little-endian ordering in the plot

## Reasoning

`reverse_bits` only changes the *display order* of the qubit subplots (useful because Qiskit's default ordering is little-endian, $q_0$ rightmost in bitstrings); it performs no mathematical transformation on the state itself.

# Answer Key — Question 9

## Correct Answer: B

`ValueError`, because `color` must be a tuple/list of exactly two entries (styling the real and imaginary bar sets)

## Reasoning

The notebook is explicit: `plot_state_city` renders two 3D bar sets — real part and imaginary part of $\rho$ — and a `ValueError` occurs if `color` is not a 2-element tuple, since each element styles exactly one of those two bar sets.

# Answer Key — Question 10

## Correct Answer: C

The transparency of the bars, where 0 is fully transparent and 1 is fully opaque

## Reasoning

`alpha` is a standard Matplotlib transparency parameter carried through by `plot_state_city`: 0 = fully transparent, 1 = fully opaque.

# Answer Key — Question 11

## Correct Answer: B

White square = positive value, black square = negative value; square size is proportional to the magnitude of that matrix entry

## Reasoning

Per the notebook: “A white square means the value is positive and a black one means negative,” and square size scales with the magnitude of the corresponding density-matrix entry.

# Answer Key — Question 12

## Correct Answer: B

The coefficient $\mathrm{Tr}(\sigma\rho)$ — the expectation value of each $n$-qubit Pauli string operator in the decomposition of $\rho$

## Reasoning

`plot_state_paulivec` plots exactly the $\mathrm{Tr}(\sigma\rho)$ coefficients from the Pauli-basis decomposition of $\rho$ shown in the notebook's formula — one bar per Pauli string, not eigenvalues or raw basis-state probabilities.

# Answer Key — Question 13

## Correct Answer: B

The provided colors are cycled/alternated in sequence across the remaining bars

## Reasoning

The notebook explicitly notes: “If you introduce a list with less colors than bars, the color of the bars will alternate following the sequence from the list” — the palette repeats, it does not error.

# Answer Key — Question 14

## Correct Answer: B

Size is proportional to the probability of that basis-state term; color encodes its phase

## Reasoning

Per the notebook comment: “size of the points is proportional to the probability of the corresponding term in the state and the color represents the phase.”

# Answer Key — Question 15

## Correct Answer: B

`show_state_phases` labels the phase value of each basis-state term directly on the qsphere; `use_degrees` displays that phase in degrees instead of the default radians

## Reasoning

These are display/formatting toggles only: `show_state_phases` annotates each term's phase on the qsphere, and `use_degrees` simply switches the unit of that phase label from radians to degrees — neither alters the underlying state.

# Answer Key — Question 16

## Correct Answer: B

`plot_gate_map` visualizes the connectivity and native gates of an actual backend object; `plot_coupling_map` lets you plot an *arbitrary* coupling map from raw `num_qubits`, coordinates, and edge list, with no backend required

## Reasoning

`plot_gate_map(backend)` draws real hardware connectivity/gates directly from a `BackendV2`-like object. `plot_coupling_map(num_qubits, qubit_coordinates, coupling_map)` takes purely abstract inputs — it is for visualizing *any* custom coupling topology, not tied to a real backend.

# Answer Key — Question 17

## Correct Answer: B

The gate and measurement error rates of the backend, overlaid on the connectivity graph

## Reasoning

`plot_error_map` augments the connectivity picture with the backend's actual gate and measurement error rates (color-coded), which `plot_gate_map` alone does not show.

# Answer Key — Question 18

## Correct Answer: B

A circuit that has already been `transpiled` for a target backend, plus that backend object; it shows how the circuit's virtual qubits were mapped onto the backend's physical qubits

## Reasoning

The notebook transpiles `ghz` against a `GenericBackendV2` first (`new_circ_lv3 = transpile(ghz, backend=backend, optimization_level=3)`) and only then calls `plot_circuit_layout(new_circ_lv3, backend)` — it visualizes the virtual-to-physical qubit mapping chosen during transpilation, which requires both a transpiled circuit and the backend.

# Answer Key — Question 19

## Correct Answer: B

The `output` argument selects the rendering backend (e.g. `text`, `mpl`, `latex`), and the `style` dict (e.g. `backgroundcolor`) customizes the appearance only when using the matplotlib renderer

## Reasoning

`output` chooses among Qiskit's multiple circuit-drawing renderers; the notebook's `style={"backgroundcolor": "#EEEEEE"}` is an `mpl`-specific styling dictionary and has no bearing on which renderer is used — no backend is required just to draw a circuit.

# Answer Key — Question 20

## Correct Answer: B

It visualizes *operation dependencies* as a Directed Acyclic Graph, a structural view distinct from `circuit_drawer`'s temporal, qubit-line view of the same circuit

## Reasoning

`dag_drawer` operates on a `DAGCircuit` (produced via `circuit_to_dag`) and shows which operations must precede others — a dependency graph — which is conceptually different from `circuit_drawer`'s left-to-right timeline of gates on qubit wires.

# Answer Key — Question 21

## Correct Answer: B

The colors of the DAG's input nodes, output nodes, and operation nodes respectively — the three structural node types in a `DAGCircuit`

## Reasoning

A `DAGCircuit` has exactly three node categories — input nodes, output nodes, and operation nodes — and the style dictionary keys map one-to-one onto them, exactly as used in the notebook.

# Answer Key — Question 22

## Correct Answer: B

qc must be transpiled with an explicit scheduling method (e.g. `scheduling_method='alap'`) and a layout method against the target backend, so it carries concrete instruction timing

## Reasoning

The notebook always runs `qc = transpile(qc, backend, scheduling_method='alap', layout_method='trivial')` before calling `draw()` — the timeline drawer needs a circuit that has already been scheduled with real timing/duration data, which only transpilation with a scheduling method provides.

# Answer Key — Question 23

## Correct Answer: B

They are alternate visual style presets offering different levels of instruction/timing detail in the rendered timeline

## Reasoning

Both are `style=` presets passed to `draw()` that change how much detail (labels, colors, annotations) the rendered timeline shows — `IQXSimple` for a cleaner view, `IQXDebugging` for a more detailed/verbose one — while the underlying scheduled circuit is identical.

# Answer Key — Question 24

## Correct Answer: B

It is deprecated as of Qiskit 1.2.0 and scheduled for removal in the 2.0 release

## Reasoning

The notebook states this explicitly: the function is deprecated as of Qiskit 1.2.0 and will be removed in the 2.0 release — an important fact for anyone relying on it for exam or production code.

# Answer Key — Question 25

## Correct Answer: B

It only accepts 1-dimensional or 2-dimensional arrays

## Reasoning

The notebook's docstring/comment is explicit: `array_to_latex` renders the LaTeX representation of a complex numpy array of dimension 1 *or* 2 — higher-dimensional tensors are not supported.

# Answer Key — Question 26

## Correct Answer: B

Quantum coherence between pairs of basis states

## Reasoning

By definition $\rho_{ij} = \alpha_i \alpha_j^*$ for $i \neq j$ captures cross-terms between basis states — i.e. coherence — while the diagonal $\rho_{ii} = |\alpha_i|^2$ carries the probabilities.

# Answer Key — Question 27

## Correct Answer: B

Strong off-diagonal terms indicate quantum superposition/coherence; zero off-diagonal terms indicate a classical mixture (no coherence between basis states)

## Reasoning

This is the reverse of option A: strong off-diagonal magnitude signals coherent superposition between basis states, while a diagonal-only density matrix (no off-diagonal terms) is the signature of a purely classical statistical mixture.

# Answer Key — Question 28

## Correct Answer: B

It extracts qubit $i$'s local, reduced single-qubit description by discarding (averaging over) its correlations with every other qubit, which is what enables one independent Bloch vector per qubit

## Reasoning

Partial trace over "all qubits except $i$" is precisely the standard operation for obtaining a subsystem's reduced density matrix; it marginalizes out correlations/entanglement with the rest of the system so that qubit $i$ alone can be represented as one point on its own Bloch sphere.

# Answer Key — Question 29

## Correct Answer: B

The reduced single-qubit states are mixed — because the two-qubit cx entangles the qubits — so each Bloch vector has length $< 1$ and lands inside the sphere

## Reasoning

The `CX` gate entangles the two qubits, so tracing out one leaves the other in a genuinely mixed reduced state ($\mathrm{Tr}(\rho_i^2) < 1$). A mixed single-qubit state always has Bloch-vector length strictly less than 1, plotting inside — not on — the sphere's surface; this is expected physics, not a bug.

# Answer Key — Question 30

## Correct Answer: B

(i) `plot_histogram`, (ii) `plot_error_map`, (iii) `dag_drawer`, (iv) `timeline_drawer`

## Reasoning

Each pairing must match the function to its documented purpose exactly: raw shot counts → `plot_histogram`; backend error rates → `plot_error_map`; operation dependency graphs → `dag_drawer`; scheduled timing → `timeline_drawer`. Only option B gets all four correct simultaneously — the other options each contain at least one mismatched function.